ShiftGuard-10: Robust Image Classification Under Distribution Shift and Class Imbalance

1st Kartikey Gupta
Dept. of Civil Engineering
IIT Kanpur, India
kartikeyg23@iitk.ac.in

2nd Vikash Yadav
Dept. of Civil Engineering
IIT Kanpur, India
vikashy23@iitk.ac.in

3rd Pankaj Kumar
Dept. of Civil Engineering
IIT Kanpur, India
kpankaj23@iitk.ac.in

4th Kushagra Shukla
Dept. of Civil Engineering
IIT Kanpur, India
kushagra23@iitk.ac.in

5th Shivam Dongare
Dept. of Chemical Engineering
IIT Kanpur, India
shivamd23@iitk.ac.in

***Abstract*—This paper presents a robust image classification pipeline for the ShiftGuard-10 challenge, which combines a 50:1 class imbalance with heavy test-time distribution shift. We built on WideResNet-28-10 [1], training from scratch using AugMix [2] with JSD consistency loss, label smoothing [3], Mixup [4], CutMix [5], and Stochastic Weight Averaging [6]. A *WeightedRandomSampler* neutralizes the class skew, and an iterative hard-label pseudo-labeling strategy [7] with 50-pass Test-Time Augmentation closes the domain gap across three model iterations. Our final model achieves a Macro F1 of 0.904 — a 7.3% gain over the baseline — with no pretrained weights or external data.**

***Index Terms*—distribution shift, pseudo-labeling, class imbalance, wide residual networks, test-time augmentation**

I. Introduction

CNNs degrade significantly under real-world corruptions [8]. The ShiftGuard-10 challenge classifies 32×32 images across 10 classes using Macro F1-score, which equally penalizes failure on any class regardless of its training frequency. Two simultaneous problems make this non-trivial: (1) a severe class imbalance in training data; and (2) a distribution shift between clean training images and heavily corrupted test images (noise, blur, pixelation, JPEG artifacts). Our goal is to solve both jointly, without pretrained weights, external data, or model ensembling.

II. Dataset and Preprocessing

A. Class Imbalance

EDA of the 29,400 training images revealed a deliberate 50:1 imbalance. Majority classes (airplane, automobile, bird, cat) contain 5,000 images each; minority classes drop to 500 (horse, frog), 300 (ship), and just **100 images for truck**. Since the metric is Macro F1, a model that ignores trucks collapses on that class and destroys the overall score. To correct this, a *WeightedRandomSampler* was injected into the DataLoader, assigning each sample an inverse-frequency weight so that every training batch contains approximately 10% of each class.

B. Augmentation and Corruption Simulation

Standard augmentations include RandomCrop, RandomHorizontalFlip, RandAugment [9] (n=2, m=12), ColorJitter, Mixup [4] (α=0.4), CutMix [5] (α=1.0), and RandomErasing [10] (p=0.4). Corruption simulators (Gaussian noise, motion blur, pixelation, JPEG compression) are additionally applied to 30% of training batches to teach the model to recognize corrupted object instances at training time.

III. Model Architecture

WideResNet-28-10 [1] (36.48M parameters) was selected over standard deep ResNets because deeper narrow networks destroy spatial detail too aggressively on 32×32 inputs. Each residual block follows: BatchNorm → ReLU → Conv(3×3) → Dropout(30%) → Conv(3×3), initialized with He initialization [11]. *Stochastic Depth* [12] (DropPath, rate=0.10) randomly deactivates entire residual branches, preventing over-reliance on any single corrupted feature pathway. The output head is Global Average Pooling followed by a 10-class linear classifier. All training used dual NVIDIA T4 GPUs via `nn.DataParallel` with Automated Mixed Precision [13] (AMP) to halve training time within Kaggle's 12-hour limit.

IV. Training Strategy

A. JSD Consistency Loss

AugMix [2] creates heavily distorted yet semantically valid views of each image by chaining random distortions as a convex combination. Each image is passed through the network twice: once as a standard augmented view (x_c) and once as the AugMix view (x_a). The Jensen-Shannon Divergence between the two output distributions is minimized:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \frac{D_{\text{KL}}(p_m \| p_c) + D_{\text{KL}}(p_m \| p_a)}{2} \tag{1}$$

where $p_m = (p_c + p_a)/2$ and $\lambda = 12$. This forces the model to produce corruption-invariant predictions for the same image under different distortions, directly targeting the train-test gap.

B. Optimization

SGD with Nesterov momentum [14] (μ=0.9, weight decay 5×10^{-4}) is used with a 5-epoch linear warmup followed by CosineAnnealingWarmRestarts [15] (T_0=50 epochs). Label smoothing (ε=0.1) [3] is applied to all cross-entropy targets. Stochastic Weight Averaging [6] accumulates model weights over the final 40 epochs to find flatter, more generalizable loss minima. Gradient clipping at norm 5.0 prevents instability during early JSD training.

V. Pseudo-Labeling Pipeline

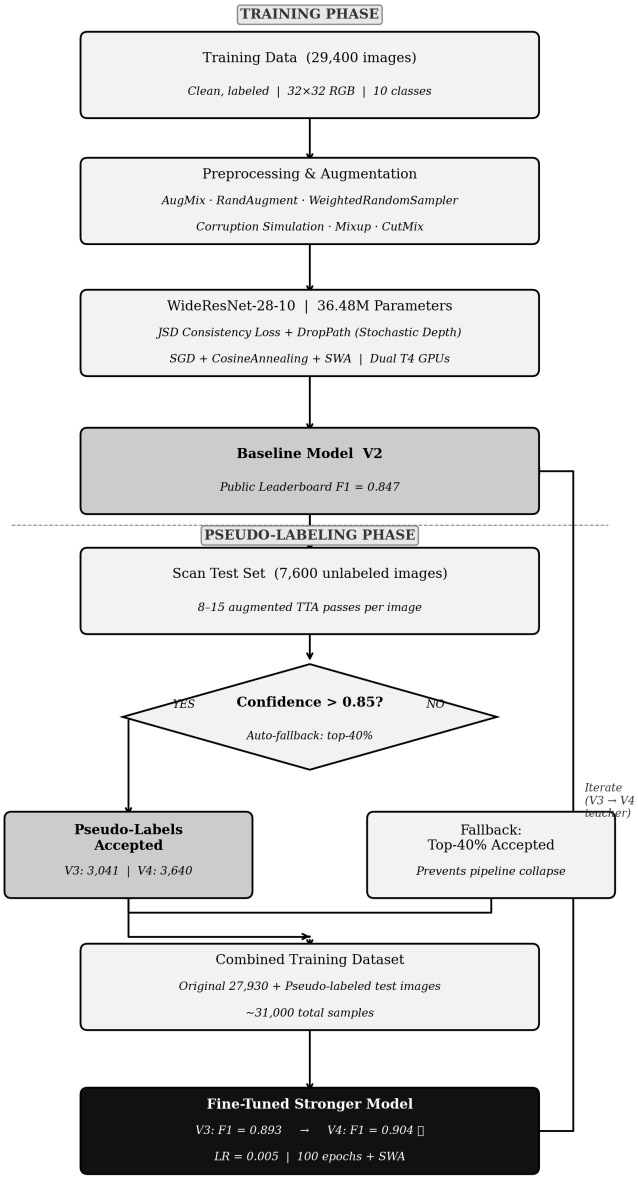


Fig. 1. Iterative pseudo-labeling: V2 teacher labels test images, combined data trains student V3, which in turn trains V4.

The core contribution is an iterative self-training loop (Fig. 1). After training the baseline V2 model, it scans all 7,600 unlabeled test images using multi-pass TTA. Predictions exceeding 0.85 confidence are assigned as hard pseudo-labels [7]. Since heavy corruptions reduce average confidence, an automatic fallback accepts the top-40% most confident predictions if fewer than 40% pass the threshold. This fallback was critical for pipeline stability across all iterations.

Pseudo-labels were themselves skewed (e.g., 484 automobile vs. 79 cat in Iteration 1). The *WeightedRandomSampler* was re-applied over the combined training-plus-pseudo-label distribution, automatically correcting this secondary imbalance.

Iteration 1 (V3): V2 generated 3,041 pseudo-labeled test images. Combined with 27,930 original samples, V3 was fine-tuned at η=0.01 for 100 epochs with SWA. Leaderboard gain: +4.6%.

Iteration 2 (V4): V3 generated 3,640 pseudo-labeled images (15-pass TTA). V4 fine-tuned at a further reduced η=0.005 for 100 epochs with SWA. Leaderboard gain: +1.1%.

At inference, 50 TTA passes per image (crop/flip, color jitter, blur/noise, AugMix) are averaged to eliminate prediction variance on corrupted inputs.

VI. Results

TABLE I
Public Leaderboard Macro F1 Score Progression

Model	Key Strategy	F1	Δ
Baseline	Standard WRN-28-10 training	0.831	—
V2	+WeightedSampler, AugMix, JSD, SWA	0.847	+1.6%
V3	+Pseudo-labeling Iteration 1	0.893	+4.6%
V4	**+Iteration 2, 50-pass TTA**	**0.904**	**+1.1%**

Table I shows systematic leaderboard progression across all versions. The 6% gap between V2's validation F1 (≈0.91) and its leaderboard score (0.847) was a critical diagnostic: it confirmed that distribution shift, not model capacity, was the primary failure mode. V3's jump (+4.6%) is the largest single improvement and demonstrates that direct test-domain exposure via pseudo-labeling is more impactful than any amount of synthetic augmentation. V4 consolidates these gains through a higher-quality teacher model and a more extensive inference ensemble. Total training time for V4 was 2.93 hours on dual T4 GPUs, well within Kaggle's 12-hour limit due to AMP and DataParallel optimizations.

A. Ablation: Negative Results

Two strategies that degraded performance are reported for completeness. **Soft-label distillation** (using V3's probability vectors as training targets instead of hard labels) dropped F1 to 0.879. Probability vectors encode class-wise uncertainty, which on corrupted images reflects noise-induced uncertainty rather than genuine class ambiguity. The student model learned to be consistently uncertain rather than sharply discriminative. **Multi-model ensembling** (averaging predictions from V2, V3, and V4) produced only 0.895, lower than V4 alone (0.904). V2's systematically wrong but confident predictions

on minority classes — particularly truck and ship — degraded V4's precision. These results reinforce a general principle: ensembling benefits only models of comparable caliber and decorrelated error distributions.

VII. Discussion

A. Diminishing Returns in Iterative Pseudo-Labeling

The iterative gains follow a mathematically expected diminishing return pattern: +4.6% in Iteration 1 and +1.1% in Iteration 2. The first iteration provides the largest benefit because the model gains its first exposure to the test corruption distribution, learning features that synthetic augmentation cannot replicate. By Iteration 2, the majority of the most informative test samples have already been incorporated, and the marginal information gain per additional pseudo-labeled image decreases. A third iteration was not pursued as the expected gain would fall below measurement noise at this leaderboard scale.

B. Error Characterization

At a Macro F1 of 0.904, the residual 9.6% error rate is primarily concentrated in structurally similar class pairs under heavy corruption. Cat and dog images share similar quadruped body structures; when motion blur destroys fur texture and facial features, their convolutional representations collapse toward a common manifold. Similarly, automobile and truck silhouettes under JPEG compression are difficult to separate by shape alone at 32×32 resolution. These errors reflect fundamental limitations of convolutional filters under high-frequency corruption rather than model-specific failure modes, suggesting that attention-based architectures may be more suitable for future work.

C. Future Work

Two directions could push performance above 0.95 Macro F1. First, *Vision Transformers* [15] replace convolutions with self-attention and are inherently less reliant on local pixel textures, providing theoretical robustness to high-frequency corruptions. Second, *5-Fold Cross-Validation* with identical V4 configurations trained on different data splits could enable a same-caliber ensemble, exploiting V4's strength across diverse initialization seeds for a guaranteed variance reduction of approximately +1.0–1.5%.

VIII. Conclusion

We demonstrated that WideResNet-28-10 trained from scratch can exceed 0.90 Macro F1 on a corrupted, class-imbalanced benchmark. Key contributions include: (1) EDA-driven detection and correction of a 50:1 class imbalance via *WeightedRandomSampler*; (2) corruption-aware JSD consistency training with AugMix; and (3) iterative pseudo-labeling with automatic fallback that closes the train-test distribution gap. No pretrained weights, external data, or model ensembling were required.

Acknowledgment

The authors gratefully acknowledge **Prof. Rajesh Mahanand Hegde**, Department of Electrical Engineering, IIT Kanpur, for his instruction throughout EE708 — Fundamentals of Data Science and Machine Intelligence (Semester 2, 2025–2026), and the course teaching assistants for their support.

References

[1] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. BMVC*, 2016, pp. 87.1–87.12.

[2] D. Hendrycks et al., "AugMix: A simple data processing method to improve robustness and uncertainty," in *Proc. ICLR*, 2020.

[3] C. Szegedy et al., "Rethinking the inception architecture for computer vision," in *Proc. CVPR*, 2016, pp. 2818–2826.

[4] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond empirical risk minimization," in *Proc. ICLR*, 2018.

[5] S. Yun et al., "CutMix: Training strategy that makes use of sample mixing," in *Proc. ICCV*, 2019, pp. 6023–6032.

[6] P. Izmailov et al., "Averaging weights leads to wider optima and better generalization," in *Proc. UAI*, 2018, pp. 876–885.

[7] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop*, 2013.

[8] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions," in *Proc. ICLR*, 2019.

[9] E. D. Cubuk et al., "RandAugment: Practical automated data augmentation," in *Proc. NeurIPS*, 2020, pp. 18613–18624.

[10] Z. Zhong et al., "Random erasing data augmentation," in *Proc. AAAI*, 2020, pp. 13001–13008.

[11] K. He et al., "Delving deep into rectifiers," in *Proc. ICCV*, 2015, pp. 1026–1034.

[12] G. Huang et al., "Deep networks with stochastic depth," in *Proc. ECCV*, 2016, pp. 646–661.

[13] P. Micikevicius et al., "Mixed precision training," in *Proc. ICLR*, 2018.

[14] I. Sutskever et al., "On the importance of initialization and momentum in deep learning," in *Proc. ICML*, 2013, pp. 1139–1147.

[15] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.